
Chapter 2 | PE 与脉动阵列

- 本章目标：实现 NPU 的计算核心。
- 内容：PE、weight-stationary 数据流、斜输入、输出对齐。
- 验收：make sim-06 全 PASS。

Chapter 2 | 从乘累加开始

- MAC 是 CNN 的最小计算单元： $\text{acc} += a * w$ 。
- int8 乘法 + int32 累加，精度与面积平衡。
- PE 是 MAC 的硬件化：输入 a 、权重 w 、部分和 psum 。

Chapter 2 | PE 接口

PE 是最小的计算单元，只做一件事： $\text{psum_out} = \text{psum_in} + a * w$ 。a 是激活，w 是权重，psum 是“部分和”——上一排算到一半的结果。

数据流的方向很重要：a 从西边进来、从东边出去；psum 从北边进来、从南边出去；权重装载后停在 PE 里不动。所有方向都固定下来，硬件连线才简单，时序才可预测。

把 PE 想成一个收银台：a 是顾客（数据）从左边进来，算完从右边出去；w 是这台收银台的单价表，装好后一直摆在台上；psum 是“到目前为止应该收多少钱”，从楼上递下来，加上这一笔再递给楼下。

每个 PE 只做一件事：金额 = 楼下载来的金额 + 顾客 x 单价。非常简单，但成千上万个 PE 同时做，就非常快。

- a_in / a_out：激活从左侧流入、向右流出。
- psum_in / psum_out：部分和从上方流入、向下流出。
- w_load / w_in：权重装载。
- 输出： $\text{psum_out} = \text{psum_in} + a_in * w$ 。

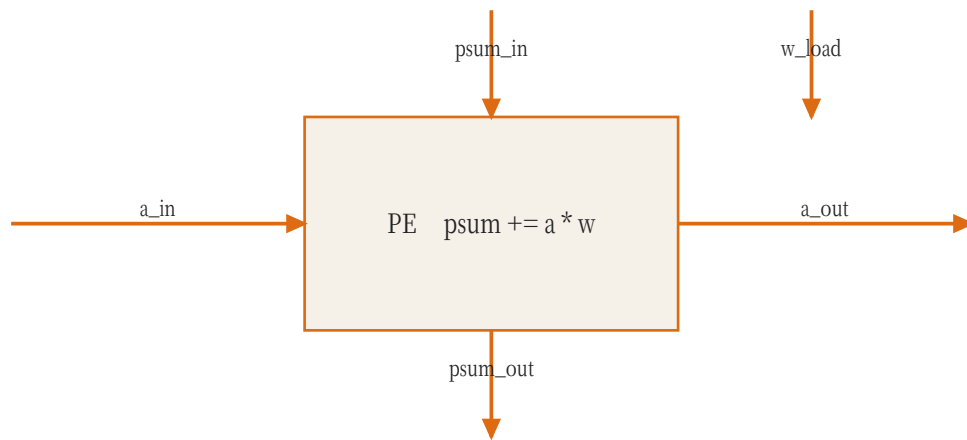


图 5：激活向右流，部分和向下流，权重装载后驻留。

Chapter 2 | 为什么叫脉动

“Systolic”源自心脏的收缩（systole）：数据像血液一样被周期性地推送到相邻 PE，每个 PE 只跟邻居通信。好处是连线短、时钟快、没有全局广播，坏处是数据必须按固定节奏流动，控制器要精确计算每一拍的地址。

心脏把血液一下一下泵出去，血液在血管里按固定节拍流动。脉动阵列也一样：数据被时钟一拍一拍“泵”进 PE，每个 PE 只和左右上下的邻居交换数据，没有全局广播。

好处是每根连线都很短，芯片可以跑很高频率；坏处是数据必须严格按节拍来，控制器要算好每一拍谁在哪儿。

- 数据像心跳一样在 PE 之间逐拍流动。
- 每个 PE 只和邻居通信，没有全局广播。
- 寄存器流水让每个周期所有 PE 都在工作。

Paper: Kung, Why Systolic Architectures?, IEEE Computer, 1982

Chapter 2 | Weight-Stationary 数据流

“Weight-Stationary”指权重停留在 PE

内、被反复使用。在卷积里，同一个权重会被很多输入像素使用，复用好；在矩阵乘里，每个权重会被一整行输入使用。

阵列中每个 PE 存一个权重，一行激活从左往右流，部分和从上往下流。这样每个周期所有 PE 都在做乘法，硬件利用率高。

“Weight-Stationary”可以翻译成“权重坐镇”。权重像印章：印章做好后放在桌上，很多张纸（输入）轮流盖，不用每次重新刻章。

阵列里每个 PE 存一个印章（权重），输入像纸一样从左边流进来，部分和像流水一样从上往下汇合。因为权重不用频繁搬动，省下的搬运时间就是加速。

- 权重先批量装载，之后停留在 PE 内。
- 激活逐拍流过阵列，部分和向下累加。
- 权重复用率 = 该权重被使用的次数。
- 卷积场景下权重复用率很高，适合这个方案。

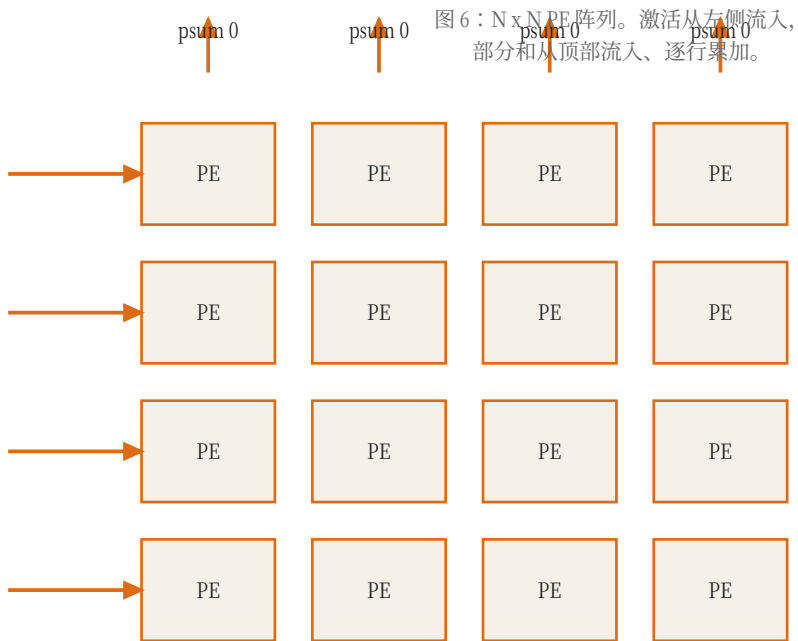


图 6：N x N PE 阵列。激活从右侧流入，
部分和从顶部流入、逐行累加。

Paper: Kung & Leiserson, Systolic Arrays (for VLSI), 1978

Chapter 2 | 斜输入

如果所有行的数据同时进入阵列，第一行处理 $k=0$ 时，第二行已经在处理 $k=1$ ，部分和就会“错位相加”。解决方法是把第 row 行的输入整体延迟 row 拍，让所有行在同一拍处理同一个 k 。

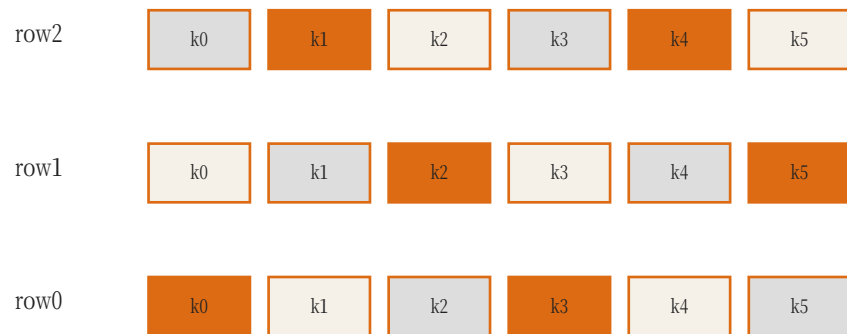
这就是“斜输入” (skewed input)。外部使用者不需要自己打 skew，阵列内部用移位寄存器自动完成。

如果三列队伍同时出发，第 1 列已经走了 1 步，第 3 列才刚起步，到终点时大家踩的点就错开了。斜输入就是让第 row 列晚 row 步出发，保证所有列在同一时刻踩到同一个节拍上。

在硬件里，“晚几步出发”就是给那一行加几个延迟寄存器。这个设计让“部分和”能一层一层正确累加。

- 阵列第 row 行的输入需要延迟 row 拍。
- 这样才能让所有行在同一拍处理同一个 k 。
- 外部不需要打 skew：阵列内部自动对齐。

图 7：row i 的输入序列整体延迟 i 拍（斜输入），保证同一列在同一拍处理同一个 k。



Chapter 2 | 输出对齐

由于激活在行内还要向右流，列 j 的结果天然比列 $j-1$ 晚一拍。如果不处理，底部输出是“斜”的，控制器取数会非常麻烦。

我们在阵列底部给第 col 列额外延迟 $N-1-col$

拍，把各列拉齐。付出几个寄存器的代价，换回“所有输出通道同一拍出现”的干净接口。

因为数据要往右流过每一列，最右边的列天然最晚算出结果，就像三条生产线长短不同，出货时间不一样。

输出对齐就是给短的生产线加一段“传送带延迟”，让三条线的成品同时到达包装台。代价只是几个寄存器，收益是控制器可以一次性拿到一整排结果。

- 列 j 的结果天然比列 $j-1$ 晚一拍。
- 在底部对第 col 列额外延迟 $N-1-col$ 拍。
- 最终所有列在同一拍输出，方便后续控制器取数。

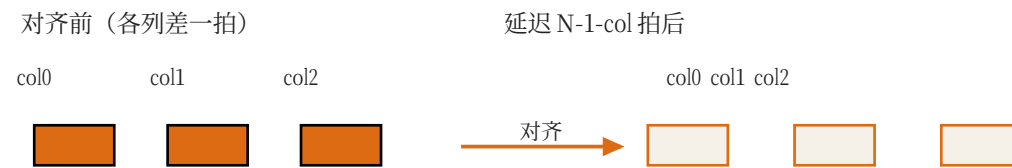


图 8：底部加移位寄存器，把不同列的斜输出拉齐到同一拍。

Chapter 2 | 时序示例

使用时序图最容易理解：w_load 高一个周期装载权重；之后每拍送一个a_data；最后一个输入送入后，还要等 N-1 拍让流水排空，才开始逐拍收到结果。

对 N=8：装载 1 拍 + 输入 8 拍 + 排空 7 拍，第一个结果在第 15 拍（0 基的第 14 拍）出现。这些数字会在 Assignment 里让你亲手推导。

把阵列想象成一条地铁：第一节车厢（第 0 行）先出发，第二节晚一站（延迟 1 拍），第三节晚两站……每一站停靠时所有车厢正好同时处理同一个 k。

所以“输入 8 拍 + 排空 7 拍”不是随便定的，而是由车厢数（N）决定的。Assignment 里会让你自己推这个公式。

- w_load 高电平装载权重。
- 之后每拍送一个 a_data 向量。
- 最后一个输入送入后再等 N-1 拍，开始逐拍输出。
- 示例：N=8，送入 8 个向量，输出 8 个结果向量。

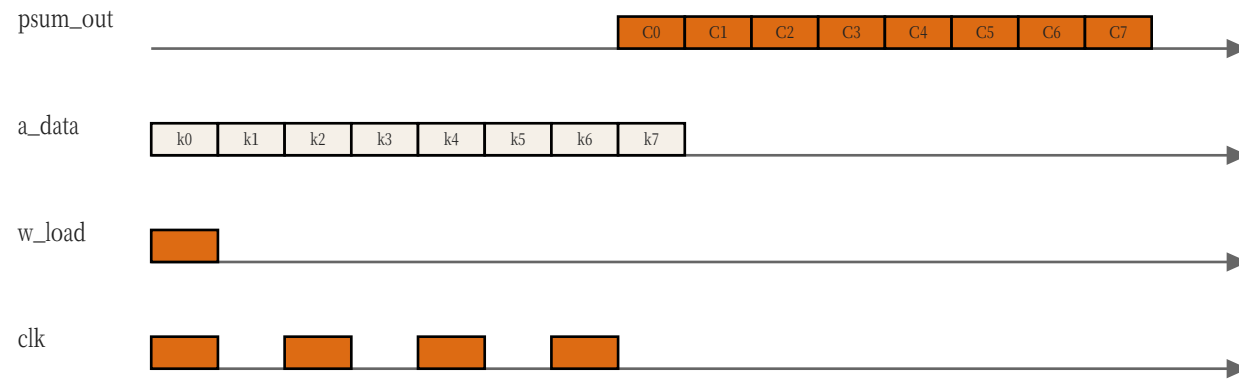


图9：送8个a_data后，输出从第7拍起逐拍出现。

Chapter 2 | 参数化设计

- N：阵列大小，8x8 起步，可扩展到 16x16。
- A_W / W_W：激活和权重位宽，默认 8。
- P_W：累加位宽，默认 32。
- 参数化让同一个 RTL 可以用于不同算力配置。

Chapter 2 | 本章任务

- 补全 PE：权重寄存器 + 有符号乘累加。
- 补全阵列：斜输入、PE 互联、输出对齐。
- 跑通 PE_tb 与 SystolicArray_tb。

Chapter 2 | 常见错误

- 忘记 signed, 导致负数乘法错误。
- w_data 位宽写成 $N \times W_W$, 应该是 $N \times N \times W_W$ 。
- 采样时机不对, 把斜输出当成错误。
- 复位不完整, 初值 x 传播到输出。

Chapter 2 | 总结

- PE 是最小的 MAC 单元。
- 脉动阵列通过数据复用把 MAC 密度拉满。
- 斜输入 + 输出对齐是正确时序的关键。
- 下一章：给阵列加上 SRAM 和控制器，做真正的卷积。

术语速查 (Glossary)

- CPU：中央处理器，负责执行指令；在本课程里是 Hack CPU。
- RAM：随机存取存储器，Hack 有 64KB，用于放数据和程序状态。
- MMIO：Memory-Mapped I/O，把外设寄存器映射到内存地址，CPU 用普通读写指令控制外设。
- 寄存器：CPU 或外设内部的小容量存储单元，通常 8/16/32 位。
- SRAM：片上静态随机存储器，速度快、容量小，NPU 用它暂存权重和特征图。
- MAC：乘累加运算： $acc = acc + a * w$ ，是 CNN 最基础的计算。
- GEMM：通用矩阵乘， $C = A \times B$ ；卷积可以转换成 GEMM。
- PE：Processing Element，处理单元；本课程里是一个 MAC 单元。
- Systolic Array：脉动阵列：PE 排成网格，数据在相邻 PE 间逐拍流动。
- Weight-Stationary：权重驻留式数据流：权重装载后停在 PE 内反复使用。
- 斜输入：把第 row 行输入延迟 row 拍，使阵列各行在同一拍处理同一个 k 。
- im2col：Image to Column，把卷积窗口展平成矩阵行，从而用 GEMM 计算卷积。
- Line Buffer：行缓冲：只保留最近若干行，供滑动窗口复用。
- FSM：有限状态机：一组状态和跳转条件，NPU 控制器用它实现调度。
- DMA：Direct Memory Access，直接内存访问，批量搬运数据。

-
- 量化：把浮点数映射到低比特整数（如 int8），并记录 scale 以便还原。
 - Scale：量化比例因子，决定一个整数单位代表多大的浮点数值。
 - Polling：轮询：CPU 反复读状态寄存器，直到外设完成。
 - argmax：取最大值的下标；分类任务里就是预测类别。
 - NPU：Neural Processing Unit，神经网络处理器，专为张量运算设计的加速器。

References / 延伸阅读

- Kung & Leiserson, "Systolic Arrays (for VLSI)", 1978.
- Kung, H.T., "Why Systolic Architectures?", IEEE Computer, 1982.
- Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA 2017.
- Sze et al., "Efficient Processing of Deep Neural Networks: A Tutorial and Survey", Proceedings of the IEEE, 2017.